

สอนมะขาม — t04_Purson มีไฟล์อะไรบ้าง แต่ละตัวมีไว้ทำไม (2026-08-29)

เขียนตามคำสั่ง "ผมอยากเข้าใจ apply_arm.py and all file in t04_Purson มีไว้ทำไมเพื่ออะไร"

เข้าใจก่อน 1 เรื่อง — ทำไมโพลเดอร์นี้เกือบทั้งหมดเป็น "ป้ายชี้ทาง"

t04 คือ t03 เวอร์ชันอัปเดต ด้านไหนไม่เปลี่ยนจาก t03 (pass 0/1/2) เราไม่ก๊อปไฟล์มา
เราวาง README เล็กๆ ชี้กลับไปไฟล์จริงแทน เหตุผล: ถ้าก๊อปมา 2 ชุด วันหนึ่งมีคนแก้ชุดเดียว
อีกชุดจะเก่าโดยไม่มีใครรู้ (เคยเกิดจริงในโปรเจกต์นี้ เรียกว่า "drift") — ของจริงมีทีเดียว
ป้ายชี้มาได้หลายป้าย

สายพานทั้งเส้น ใน 1 ย่อหน้า

บ้าน 1 หลัง = PDF 20-30 หน้า → **pass 0** (AI ดูทีละหน้า ตอบว่าหน้านี้เป็นแบบอะไร) →
pass 1 (โค้ดตัดรูปแยกโพลเดอร์ตามคำตอบ pass 0) → **pass 1.5** (ตา CV ไม่ใช่ AI
เอาไอคอนตัวอย่างไปดูหาทั้งภาพ นับว่ามีเสา/ฐานราก/คานก็จุด อยู่ตรงไหน) →
pass 2/2.4 (AI อ่านแบบจริง — 2.4 คือแบบมีโพยจาก CV เปะท้าย prompt) →
pass 2.5 (CV กวาดซ้ำด้วย template ที่เพ็งเจอในหน้านั้นเอง เก็บตกตัวที่คลั่งกลางพลาด) →
pass 3 (AI ถอดระยะ/เหล็ก จากบัญชีที่ยืนยันแล้ว — เขียน prompt เสร็จ ยังไม่เคยรัน) →
เข้า Constaint คำนวน BOQ/BBS (pass 4 หนำรอง ยกเลิกแล้ว 2026-08-29 — มะขามสั่งเอง
"หน้าพวกนั้นไม่จำเป็น")

ไฟล์ทีละตัว

README.md

ดัชนีของทั้งโพลเดอร์ — อ่านไฟล์นี้ไฟล์เดียวรู้ว่าอะไรอยู่ไหน สถานะอะไรเสร็จ/ไม่เสร็จ
ที่มาชื่อ Purson ก็อยู่ในนี้ (มะขาม commit ว่า "อยากได้ชื่อเห่ๆ อะมันตัวนำโชค")

RUNBOOK.md

คู่มือ "ทำตามทีละบรรทัดแล้วจบ" สำหรับคืนยิง GPU จริง — เช็คของก่อนเริ่ม, กติกาเข้า Vast
(การรันถูกก่อน / venv แยก / ดึงผลก่อน destroy), คำสั่งยิง 2 แขน, วิธีอ่านผลตัดสินด้วยเลข
ต่างจาก README ตรงที่ README = เข้าใจ, RUNBOOK = ลงมือ

WORKFLOW_INTERN.md (ใหม่ 2026-08-29)

คู่มือคนใหม่/intern เดินบ้าน 1 หลังผ่านทุกด้าน — บอกทั้งคำสั่ง วิธีใช้ว่าผ่าน
และ "ช่องว่างที่ยังไม่พร้อม" (จะได้รู้ว่าอะไรคือข้อจำกัดระบบ ไม่ใช่ตัวเองทำผิด)

pass0/README.md → ซิปไป `tune_ai/t03/pass0_classify/`

prompt คัดหน้า — ด้านอันตรายสุดของสายพาน เพราะคัดผิด = หน้านั้นหายทั้งสาย
(เคยเกิดจริง: โครงหลังคาโดนติดป้ายผิด 8 ใน 11 หลัง คานหลังคาหายจาก BOQ หมด)
จุดสำคัญ: pass 0 ตอบเป็น "subtask ปลายทาง" (plan_beam, section, ...) ไม่ใช่ pattern
เพราะ pass 1 ต้องรู้ว่าเอารูปไปวางโพลเดอร์ไหน

pass1/README.md → ซิปไป `tune_ai/t03/pass1_organize/organize.py`

โค้ดล้วน ไม่ใช่ AI — อ่านผล pass 0 แล้วตัดรูปจริง (หน้าที่มี 2-3 รูปตัดแยกเป็นรูปละไฟล์)
แยกลงโพลเดอร์ตาม subtask ทำไม่ต้องตัด: โควตาความละเอียดต่อรูปคงที่ ตัดครั้งมา =
ความชัดต่อพื้นที่เพิ่มเท่าตัวฟรี

pass1.5_cv/README.md → ซิปไป `tools/cv_scan.py`

"ตา CV" — ไม่ใช่ AI เลย มั่วไม่เป็น เอาไอคอนตัวอย่างจากคลัง (tools/templates/) ไปดูหา
ทั้งภาพ ได้ 3 ไฟล์ต่อหน้า: พิกัด JSON + ภาพมาร์คเลข + ข้อความ hint
ภาพมาร์คเลขตอนนี้ไว้ให้คนดูอย่างเดียว (เดิมจะป้อนโมเดลในแขน 2.4b — ยกเลิกแล้ว)

pass2/README.md → ซิปไป `tune_ai/t03/pass2_used/` (prompt 7 ตัว)

prompt อ่านแบบจริงของแต่ละ subtask: gridline (สร้าง grid master) / plan (ผัง 4 ชนิด) /
section (รูปตัดสเปกเหล็ก) / schedule (ตารางสรุป) / notes (ข้อกำหนดโครงการ) /
material_list (BOQ) / soil_boring_log (เจาะดิน) — ทุกตัว prepend _common.md
(กติการวม + **พจนานุกรมไทย→field** ที่ย้ายไปไว้ต้น prompt แล้ว 2026-08-29 ตามหลัก DIP
ที่มะขามไปค้นมา) — 2026-08-29 ล้างอักขระ markdown ออกหมดทุกตัวแล้ว (. _ * # : ; \ | —
นอก JSON/ชื่อไฟล์) กัน AI หลอน ตามคำสั่งมะขาม

pass2.4_hint/prompt.md

"สัญญา" ของ hint block — บอกว่าโพยที่แปะท้าย prompt ต้องมีอะไร (จำนวนจุดต่อชนิด +
กฎกันหลอน 4 ข้อ + คำสั่ง cv_mark) และห้ามมีอะไร (ตัวเลขระยะเด็ดขาด — ส่งเข้าไป
เท่ากับสอนโมเดลแหกกฎ "ห้ามเดาระยะจากรูป" ด้วยมือเราเอง)
ตัวโค้ดที่สร้างข้อความจริงคือ cv_hint_text() ใน tools/cv_scan.py — ไฟล์นี้เป็นแค่สัญญา
กันสองที่ drift กัน

แขนทดลองเหลือ 2: แขน 2 (ไม่มีโพย = ตัวคุม) vs แขน 2.4a (มีโพยข้อความ)

แขน 2.4b (ภาพมาร์คเลข) มะขามสังยกเล็ก 2026-08-29 "ใส่แต่ hint พอ"

pass2.4_hint/assembled_example.md

ตัวอย่าง "ของจริงประกอบเสร็จ" — เอา prompt + hint จาก sidecar จริงของบ้าน_ใหญ่_1ชั้น_04 หน้า12 มาแปะให้ดูที่ละพาร์ท จะเห็นว่าโมเดลได้รับอะไรจริงๆ ไม่ต้องจินตนาการจากสัญญา

pass2.5_harvest/README.md → ซิปไป tools/cv_scan.py (--pass25)

self-harvest — เอาไอคอนที่ CV เพ็งเจอในหน้านั้นเอง มาเป็น template ใหม่ กวาดซ้ำแบบเข้ม

(0.90) หาตัวที่คลั่งกลางจับข้ามซีรีส์ไม่ติด พิสูจน์ด้วยตาแล้ว 2 หลัง ตรง GT เป๊ะ

ข้อจำกัดโดยหลักการ: ภูได้เฉพาะชนิดที่คลั่งจับติดอย่างน้อย 1 จุด — หน้าที่ CV เจอ 0 ช่วยไม่ได้

ผลลัพธ์ (ไฟล์ลงท้าย 25) = บัญชีตั้งต้นของ pass 3

pass3_takeoff/prompt.md

prompt ถอดระยะ/หลัก — ป้อนบัญชี element ที่ยืนยันแล้ว (จาก 2.5) + ภาพ + grid master

กฎหลัก: โมเดลห้ามลบ element ในบัญชี และเราไม่เชื่อว่า prompt อย่างเดียวกันได้ —

tools/merge_guard.py บังคับด้วยโค้ด: ตัวไหนหายใส่กลับเป็น stub ติดธงให้คนดู

สถานะ: เขียนเต็ม ยังไม่เคยรัน (ตั้งใจรอผลแขนทดลองก่อน — ถ้าโมเดลไม่อ่าน hint

pass 3 ทั้งก่อนต้องคิดใหม่)

pass4/README.md

✗ ยกเลิกแล้ว (2026-08-29 มะขามสัง) — เดิมคือหน้ารอง (สถาปัตย์/ไฟฟ้า/สุขาฯ/ปก/สารบัญ)

prompt เดิมยังเก็บอยู่ที่ t03/pass4_unused/ เจๆ ไม่ลบ แะไม่อยู่ในสายพาน

apply_arm() — ตัวสลับแขนทดลอง (อยู่ใน infer_house_t03.py ไม่ใช่ไฟล์แยก)

ไม่มีไฟล์ชื่อ apply_arm.py — มันคือฟังก์ชัน `apply_arm()` ใน

`tune_ai/t03/data_before_tune/infer_house_t03.py` (ตัวรันวัดผลโมเดล)

มีไว้ทำไม: การทดลอง 2 แขนต้องยิง "หน้าเดียวกัน prompt เดียวกัน โมเดลเดียวกัน"

ต่างกันแค่มี/ไม่มีโพย — ถ้าให้คนไปแก้ prompt เองทีละรอบ เดียวก็แก้พลาดแล้วผลเทียบกันไม่ได้

`apply_arm()` คือคนกลางที่รับ content เดิม แล้ว "เติมของ" ตามแขนที่เลือกด้วย flag `--arm`

คนรันไม่ต้องแตะ prompt เลย

ทำงานยังไง (หลังตัด 2.4b แล้ว เหลือเรียบมาก):

1. แขน "2" → คีนของเดิมเป๊ะ ไม่แตะอะไร (ตัวคุม)

2. แชน "2.4a" → ไปหาไฟล์โพย <ชื่อรูป>_hint.txt ในโฟลเดอร์ sidecar (--cv-dir)

เจอ → แปะข้อความโพยต่อท้าย prompt แล้วบันทึก hint_used: true

3. หาไม่เจอ → คีนของเดิม + hint_used: false — แถวนั้นกลายเป็นแชน 2 โดยพฤตินัย

ไม่ error ไม่ดา ไม่สร้างโพยเปล่า (สำคัญ: บ้านที่คลัง template ยังไม่รู้จัก CV จะนับได้ 0

ก็จะมีโพยเอง — ระบบไม่พังแค่บันทึกไว้ให้เห็น)

อ่านผลยังง: ใน results ทุกแถวมี hint_used — แชน 2.4a ของหน้าที่มี sidecar

ต้องเป็น true ถ้า false แปลว่า path --cv-dir ผิด หยุดเช็คก่อนยิงต่อ (เปลือง GPU ฟรี)

ศัพท์ที่โผลบ่อย

- **sidecar** = ไฟล์ผลพลอยได้ที่วางข้างรูป (_cv.json / _hint.txt / _marked.png)
- **hint / โพย** = ข้อความบอกโมเดลว่า CV เห็นอะไรก็จุด — เป็นคำใบ้ ไม่ใช่คำตอบ

(กฎ 4 ข้อกันโมเดลเชื่อโพยเกินไป)

- **arm / แชน** = เส้นไขการทดลอง 1 แบบ — เทียบกันเพื่อตอบ "โพยช่วยจริงไหม"
- **GT (ground truth)** = เฉลยที่คนตรวจแล้ว ใช้ให้คะแนนโมเดล
- **recall_printed** = เปอร์เซ็นตำราที่พิมพ์บนแบบจริงที่โมเดลอ่านเจอ — ตัวชี้วัดหลัก
- **template / คลัง** = รูปไอคอนตัวอย่าง (tools/templates/) ที่ CV เอาไปดูหาทั้งภาพ